

实验六：基于 FUSE 的可读写内存文件系统（学生版）

1. 实验目标

1. 理解 Linux VFS 如何通过 FUSE 将文件系统请求转发给用户态程序。
2. 掌握 FUSE 中 `getattr`、`readdir`、`create`、`open`、`read`、`write`、`truncate`、`unlink` 等核心回调的职责。
3. 实现一个最小可用的可读写内存文件系统，能够在挂载目录中创建、读取、覆盖写入、截断和删除文件。
4. 分析内存文件系统与真实磁盘文件系统在元数据、数据持久性、错误处理和一致性方面的差异。

2. 实验环境与编译运行要求

本实验在 Ubuntu Linux 环境中完成，使用 FUSE 2.x API。

安装依赖：

```
sudo apt update
sudo apt install libfuse-dev pkg-config
```

确认 FUSE 版本：

```
pkg-config --modversion fuse
```

3. 实验任务：实现最小可读写内存文件系统

你需要补全 `memfs.c` 中所有 `TODO`。本实验只要求支持根目录 `/` 下的普通文件，例如 `/a.txt`、`/note`。不要求实现多级目录、符号链接、文件权限继承、真实磁盘持久化或并发锁。

文件系统运行期间，所有文件元数据和文件内容都保存在进程内存中。FUSE 进程退出后，创建的文件和写入的数据都会消失。

3.1 功能要求

必须支持以下命令或等价操作：

```
ls -l ~/memfs_mount
echo "hello" > ~/memfs_mount/a.txt
cat ~/memfs_mount/a.txt
printf "XY" >> ~/memfs_mount/a.txt
truncate -s 3 ~/memfs_mount/a.txt
rm ~/memfs_mount/a.txt
```

必须实现以下行为：

1. `ls` 能列出当前已经创建的文件。
2. `cat` 能按文件当前内容读取数据。

3. `echo > file` 能创建或覆盖文件内容。
4. `printf >> file` 能在文件末尾追加内容。
5. `truncate -s N file` 能放大或缩小文件。
6. `rm file` 能删除文件，删除后再次访问应返回不存在。
7. 单个文件最大容量为 `MAX_FILE_SIZE` 字节，超出容量的写入应返回错误。
8. 文件名只需要支持根目录下的一级普通文件名，不支持 `/dir/a.txt`。

4. 待补全框架代码

将下面代码保存为 `memfs.c`，补全所有 `TODO` 后编译运行。

4.1 回调函数作用速览

在补代码前，先理解：本实验真正要接住的是 FUSE 回调。辅助函数只是为了让这些回调更好写，阅读代码时应先看回调，再回头实现辅助函数。

函数	什么时候会被调用	你要实现的核心含义
<code>memfs_getattr</code>	<code>ls -l</code> 、 <code>cat</code> 、 <code>open</code> 前的路径检查	告诉 VFS：这个路径是目录、普通文件，还是不存在
<code>memfs_readdir</code>	执行 <code>ls</code> 时	把当前存在的文件名交给 VFS
<code>memfs_create</code>	对不存在的文件执行 <code>echo > file</code> 等操作时	在内存文件表中新建一个空文件
<code>memfs_open</code>	打开已有文件时	判断文件是否存在，允许后续读写
<code>memfs_read</code>	<code>cat file</code> 或程序调用 <code>read</code> 时	按 offset 从文件内容中复制一段数据给调用者
<code>memfs_write</code>	<code>echo</code> 、 <code>printf >></code> 或程序调用 <code>write</code> 时	按 offset 写入数据，并更新文件大小
<code>memfs_truncate</code>	<code>truncate -s</code> 、重定向覆盖、某些编辑器保存时	改变文件长度，可能缩短文件或清零扩展区域
<code>memfs_unlink</code>	<code>rm file</code> 时	从文件表中删除文件

这些回调合在一起，才构成一个最小文件系统：路径检查由 `getattr` 回答，目录列表由 `readdir` 回答，文件生命周期由 `create/open/unlink` 处理，文件内容变化由 `read/write/truncate` 处理。

为了避免每个回调都重复写查找和路径判断，框架最后还提供了三个辅助函数：

辅助函数	为什么需要它
<code>find_file</code>	多个回调都需要按文件名找到内存文件表中的槽位
<code>find_free_slot</code>	<code>create</code> 需要为新文件找到空闲槽位

辅助函数	为什么需要它
<code>path_to_name</code>	多个回调都需要把 FUSE 路径 <code>/a.txt</code> 转成本实验内部文件名 <code>a.txt</code>

4.2 框架代码

```
#define FUSE_USE_VERSION 26

#include <fuse.h>
#include <errno.h>
#include <fcntl.h>
#include <stddef.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>

#define MAX_FILES 64
#define MAX_NAME_LEN 127
#define MAX_FILE_SIZE 4096

typedef struct {
    int used;                // 当前槽位是否已经保存了一个文件
    char name[MAX_NAME_LEN + 1]; // 文件名，不包含路径开头的 '/'
    char data[MAX_FILE_SIZE];    // 文件内容缓冲区
    size_t size;                // 当前文件实际大小，单位为字节
} mem_file_t;

static mem_file_t files[MAX_FILES];

// 下面先声明辅助函数。它们的实现放在 FUSE 回调之后。
// 这样你可以先阅读“文件系统要拦截哪些操作”，再理解每个回调需要哪些共同工具。
static int find_file(const char *name);
static int find_free_slot(void);
static int path_to_name(const char *path, const char **name);

// FUSE 回调的返回值约定：
// 返回 0：操作成功。
// read/write 返回正数：实际读/写的字节数。
// 返回负的 errno：操作失败，并告诉 VFS 失败原因。
// 常用错误码：
// -ENOENT：没有这个文件或目录。
// -EEXIST：要创建的文件已经存在。
// -ENOSPC：没有可用空间；在本实验中通常表示 files[] 没有空槽位。
// -ENAMETOOLONG：文件名太长；在本实验中通常表示超过 MAX_NAME_LEN。
// -EFBIG：文件太大；在本实验中通常表示超过 MAX_FILE_SIZE。
// -EINVAL：参数非法，例如 offset 或 size 不合理。
// -EISDIR：把目录当作普通文件处理。
// -ENOTDIR：路径中间某一层不是目录，进阶任务中常见。
// -EMPTY：目录不是空的，进阶任务中删除目录时常见。
// -EACCES：权限不允许。

// getattr 回答 VFS 的“这个路径是什么”问题。
```

```

// 输入:
// path 表示 VFS 正在查询的路径, 例如 "/" 或 "/a.txt"。
// stbuf 是输出参数, 你要把查询结果写进这个 struct stat 结构体。
// 返回:
// 路径存在时返回 0, 并正确填写 stbuf。
// 路径不存在或不支持时返回上面列出的负错误码, 例如 -ENOENT。
// 本实验会用到的 struct stat 成员:
// st_mode : 文件类型和权限。文件类型可选 S_IFDIR (目录) 或 S_IFREG (普通文件);
//          权限可选 0755、0644 等, 使用“文件类型 | 权限”组合。
//          权限数字是八进制: 三位分别表示所有者、同组用户、其他用户。
//          每一位中 4 表示可读, 2 表示可写, 1 表示可执行/可进入目录。
//          0755 表示所有者可读写执行, 其他人可读和执行, 常用于目录。
//          0644 表示所有者可读写, 其他人只读, 常用于普通文件。
// st_nlink : 链接数。目录在本实验中可设为 2, 普通文件可设为 1。
// st_size : 文件大小, 单位为字节。目录不需要设置; 普通文件应使用当前文件大小。
static int memfs_getattr(const char *path, struct stat *stbuf) {
    memset(stbuf, 0, sizeof(struct stat));

    // TODO 1: 根目录 "/" 分支。
    // 当前条件: path 表示根目录。
    // 目标: 把 stbuf 填成“目录”的元数据。

    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;

    // TODO 2: 使用 find_file(name) 在 files[] 中查找该文件。
    // 当前条件: path_to_name 已经把 "/a.txt" 转成 name = "a.txt"。
    // 目标: 确认这个普通文件是否真的存在; 找不到则表示路径不存在。

    // TODO 3: 普通文件元数据分支。
    // 当前条件: 文件已经存在, 并且你已经拿到了它在 files[] 中的下标。
    // 目标: 把 stbuf 填成“普通文件”的元数据。
}

// 回答 VFS 的“这个目录下面有哪些名字”问题。
// 执行 ls 时会调用 readdir; 你需要把当前 used 的文件名逐个交给 filler。
static int memfs_readdir(const char *path, void *buf, fuse_fill_dir_t filler,
                        off_t offset, struct fuse_file_info *fi) {
    (void) offset;
    (void) fi;

    if (strcmp(path, "/") != 0)
        return -ENOENT;

    // 目录通常包含 "." 和 ".." 两个特殊目录项:
    // "." 表示当前目录;
    // ".." 表示父目录。
    // filler 的作用是把一个名字加入 readdir 返回给 VFS 的目录列表。
    filler(buf, ".", NULL, 0);
    filler(buf, "..", NULL, 0);

```

```

// TODO 4: 在这里遍历 files 数组。
// 对每一个正在使用的文件槽位，都需要调用 filler 把文件名加入目录列表。
// filler 的参数形式是：
// filler(目录缓冲区，要加入的目录项名称，目录项的 stat 信息，偏移量);

    return 0;
}

// 创建一个新的普通文件。
// 当用户对不存在的文件执行 echo "... " > file 时，FUSE 可能调用 create。
// 本实验中新文件的内容一开始为空，后续内容由 write 写入。
static int memfs_create(const char *path, mode_t mode, struct fuse_file_info *fi) {
    (void) mode;
    (void) fi;

    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;

    // TODO 5: 调用 find_file(name) 检查同名文件是否已经存在。
    // 如果已经存在，返回相应的错误码

    // TODO 6: 调用 find_free_slot() 找一个空槽位，把返回值保存为 idx。
    // 如果 idx < 0，说明文件数量已经达到 MAX_FILES，返回相应错误码。

    // TODO 7: 初始化 idx 对应的新文件槽位。
    // 目标：让它成为一个“已存在但内容为空”的普通文件。
    // 初始化成功后返回 0。
}

// 打开一个已经存在的文件。
// 基础任务中只需要确认文件存在；如果不存在，VFS 应得到 -ENOENT。
static int memfs_open(const char *path, struct fuse_file_info *fi) {
    (void) fi;

    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;

    // TODO 8: 调用 find_file(name)。
    // 如果找到文件，返回 0，表示 open 成功。
    // 如果没有找到，返回相应错误码。
}

// 从文件中读取一段数据。
// cat file 不一定一次读完整个文件，VFS 会带着 offset 和 size 多次请求。
// 你需要只返回文件实际范围内的数据，并用返回值告诉 VFS 读到了多少字节。
static int memfs_read(const char *path, char *buf, size_t size, off_t offset,
                     struct fuse_file_info *fi) {

```

```

(void) fi;

const char *name;
int ret = path_to_name(path, &name);
if (ret < 0)
    return ret;

// TODO 9: 实现普通文件读取。
// 目标: 根据 offset 和 size, 从文件当前内容中返回一段有效数据。
// 读取规则:
// 1. 先确认文件存在。
// 2. offset合法性判断。
// 3. 请求的size比真实剩余的size大时, 做相应操作。
// 4. 将实际可读取的字节复制到 buf 中。
// 5. 成功时返回实际读取的字节数。
}

// 向文件中写入一段数据。
// echo、printf >>、编辑器保存都会触发写入, 但写入位置由 offset 决定。
// 如果写入超过原文件末尾, 需要扩展文件大小。
static int memfs_write(const char *path, const char *buf, size_t size, off_t offset,
                      struct fuse_file_info *fi) {
    (void) fi;

    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;

    // TODO 10: 实现普通文件写入。
    // 目标: 根据 offset 把传入数据写入文件内容, 并在必要时扩展文件大小。
    // 写入规则:
    // 1. 先确认文件存在。
    // 2. offset 合法性判断。
    // 3. 写入后的文件范围不能超过 MAX_FILE_SIZE。
    // 4. 将传入的字节写入文件内容中 offset 对应的位置。
    // 5. 如果写入后文件变长, 更新文件当前大小。
    // 6. 成功时返回实际写入的字节数。
}

// 改变文件长度。
// truncate -s N 会直接触发它; shell 覆盖重定向和某些编辑器保存也可能依赖它。
// 缩短文件会丢弃末尾内容; 放大文件时, 新增区域应当被清零。
static int memfs_truncate(const char *path, off_t size) {
    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;

    // TODO 11: 实现文件长度调整。
    // 目标: 把文件大小改为参数 size。
    // 调整规则:
    // 1. 先确认文件存在。

```

```

    // 2. size 合法性判断。
    // 3. 如果文件放大，新增区域应按空字节处理。
    // 4. 更新文件当前大小。
    // 5. 成功时返回 0。
}

// 删除一个普通文件。
// rm file 会触发 unlink；删除后，该文件不应再能被 getattr/open 找到，
// 也不应继续出现在 readdir 的目录列表中。
static int memfs_unlink(const char *path) {
    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;

    // TODO 12: 实现文件删除。
    // 目标：让该文件不再存在，也不再出现在 readdir 的目录列表中。
    // 删除规则：
    // 1. 先确认文件存在。
    // 2. 将该文件槽位恢复为未使用状态。
    // 3. 成功时返回 0。
}

// ===== 辅助函数：服务于上面的 FUSE 回调 =====

// 按文件名在内存文件表中查找文件。
// getattr/open/read/write/truncate/unlink 都会用它定位文件槽位。
static int find_file(const char *name) {
    // TODO 13: 在文件表中查找名为 name 的有效文件。
    // 有效文件指 used 为真的槽位；文件名应按字符串内容判断是否相同。

    // TODO 14: 找到时返回文件所在槽位下标；找不到时返回 -1。
}

// 为新文件寻找一个空闲槽位。
// create 回调会用它决定新文件可以放到 files[] 的哪个位置。
static int find_free_slot(void) {
    // TODO 15: 在文件表中寻找一个尚未使用的槽位。

    // TODO 16: 找到时返回槽位下标；如果文件表已满，返回 -1。
}

// 将 FUSE 传入的路径转换成本实验内部使用的文件名。
// 本实验只支持根目录下的普通文件，所以 "/a.txt" 合法，
// "/" 是根目录，"/dir/a.txt" 这种多级路径不属于基础任务范围。
static int path_to_name(const char *path, const char **name) {
    // TODO 17: 检查路径格式。
    // 本实验只接受 "/filename"；根目录、多级路径和空文件名都不应被当作普通文件。

    // TODO 18: 检查文件名长度。
    // 文件名超过 MAX_NAME_LEN 时，应返回文件名过长对应的错误码。

    // TODO 19: 如果前面检查都通过，就通过输出参数 name 返回文件名部分，并返回 0。
}

```

```

}

static struct fuse_operations memfs_oper = {
    .getattr = memfs_getattr,
    .readdir = memfs_readdir,
    .create = memfs_create,
    .open = memfs_open,
    .read = memfs_read,
    .write = memfs_write,
    .truncate = memfs_truncate,
    .unlink = memfs_unlink,
};

int main(int argc, char *argv[]) {
    return fuse_main(argc, argv, &memfs_oper, NULL);
}

```

5. 预期运行结果与观察重点

普通的 `echo`、`cat`、`rm` 在 FUSE 挂载点里看起来和普通目录很像，这是正常的。文件系统的目标就是让普通程序不用关心底层实现。为了确认自己访问的是 `memfs`，需要观察“挂载前后”和“卸载后”的差异。

先将上面的框架代码保存为 `memfs.c`，补全 TODO 后编译：

```
gcc -Wall memfs.c `pkg-config fuse --cflags --libs` -o memfs
```

如果编译成功，会生成可执行文件 `memfs`。

5.1 挂载前后对比

先准备一个挂载点，并在还没有挂载时放入一个普通磁盘文件：

```
mkdir -p ~/memfs_mount
echo "real disk file" > ~/memfs_mount/before_mount.txt
ls -l ~/memfs_mount
```

此时看到的内容来自原来的磁盘目录。

然后启动自己的文件系统。因为这里故意在挂载点中保留了 `before_mount.txt` 用于对比，所以需要加上 `-o nonempty` 允许 FUSE 挂载到非空目录：

```
./memfs -o nonempty ~/memfs_mount
```

`-o nonempty` 只用于本节的观察演示。平时使用 FUSE 文件系统时，仍建议挂载到空目录，避免把原目录内容暂时遮住后产生误解。

执行下面这条命令，可以看到我们指定的那个路径已经成为 FUSE 挂载点。

```
mount | grep memfs_mount
```


5.2 基础功能验证

在另一个终端执行：

```
echo "hello" > ~/memfs_mount/a.txt
cat ~/memfs_mount/a.txt
printf " world" >> ~/memfs_mount/a.txt
cat ~/memfs_mount/a.txt
truncate -s 5 ~/memfs_mount/a.txt
cat ~/memfs_mount/a.txt
rm ~/memfs_mount/a.txt
ls -l ~/memfs_mount
```

你应能观察到：

1. 新文件会出现在 `ls` 输出中。
2. `cat` 能读出刚写入的内容。
3. 追加写会改变文件长度。
4. 截断后文件内容只保留指定长度。
5. 删除后文件从目录列表中消失。

5.3 卸载后观察

卸载文件系统：

```
fusermount -u ~/memfs_mount
ls -l ~/memfs_mount
```

你应能观察到：

1. `before_mount.txt` 又出现了，因为原来的磁盘目录重新暴露出来。
2. 在 `memfs` 中创建的 `a.txt` 不会保留在这个目录里，因为基础版文件内容只存在于 FUSE 进程内存中。

这个现象说明：挂载不是把文件写进原目录，而是让某个路径暂时由另一个文件系统实现接管。

5.4 前台调试观察

如果想直接看到普通命令触发了哪些 FUSE 回调，以前台调试模式运行：

```
./memfs -f -d -o nonempty ~/memfs_mount
```

另开一个终端执行：

```
echo "hello" > ~/memfs_mount/a.txt
cat ~/memfs_mount/a.txt
rm ~/memfs_mount/a.txt
```

你应能在运行 `./memfs -f -d` 的终端里看到 FUSE 请求日志。结合这些日志，再回到代码中观察 `getattr`、`create`、`write`、`read`、`unlink` 等回调是如何被普通命令触发的。

6. 实验思考题

1. `echo "hello" > a.txt` 这条命令可能触发哪些 FUSE 回调？请结合前台调试模式 `-f -d` 的日志，说明这些回调为什么按这样的顺序出现。
2. 为什么 `getattr` 中返回的 `st_mode` 和 `st_size` 会影响后续 `open`、`read`、`write` 的表现？如果文件大小返回错误，用户态程序可能观察到什么异常？
3. 本实验把文件表和文件内容都放在进程内存中。请分析：这和真实磁盘文件系统 `inode`、目录项、数据块分别有什么对应关系？又有哪些关键能力没有实现？
4. `read` 和 `write` 都带有 `offset` 参数。为什么文件系统回调不能假设每次读写都是从文件开头开始？追加写、覆盖写、截断后再写分别会怎样影响 `offset` 和文件大小？
5. 本实验没有实现并发保护。如果两个终端同时向同一个文件写入内容，可能出现哪些竞争问题？如果要修复，你会给哪些共享数据加锁，锁的粒度如何选择？

7. 自选进阶任务（二选一）

两个进阶任务都要求在基础任务完成后实现。挑战 A 是对当前内存文件系统的结构性增强，要求你把“只有根目录下普通文件”的模型扩展成真正的目录树；挑战 B 则要求你把 FUSE 用作真实目录外面的一层透明变换。选择其中一个完成即可。

挑战一：多级目录内存文件系统

在基础内存文件系统上增加任意多级目录支持，例如 `/docs/week1/a.txt`、`/tmp/cache/data.bin`。目录和普通文件都仍然保存在程序内存中；程序退出后，所有目录和文件都消失。

要求：

1. 支持 `mkdir` 创建多级路径中的目录，例如先创建 `/docs`，再创建 `/docs/week1`。
2. 支持在任意已有目录中创建、读取、写入、截断、删除普通文件。
3. `readdir` 只能列出当前目录的直接子项，不能把孙目录中的内容一起列出。
4. 不允许删除非空目录。
5. 同一目录下不允许重名；不同目录中可以存在同名目录或同名普通文件。
6. 需要重新设计文件元数据，使目录和普通文件能够区分，并能记录父子关系。
7. 在报告中说明路径解析如何逐级查找 `/docs/week1/a.txt`，以及当中间路径不存在或中间路径不是目录时应返回什么错误。

验收建议：

```
mkdir ~/memfs_mount/docs
mkdir ~/memfs_mount/docs/week1
mkdir ~/memfs_mount/docs/week2
echo "root" > ~/memfs_mount/a.txt
echo "week1" > ~/memfs_mount/docs/week1/a.txt
echo "week2" > ~/memfs_mount/docs/week2/a.txt
ls ~/memfs_mount/docs
cat ~/memfs_mount/docs/week1/a.txt
cat ~/memfs_mount/docs/week2/a.txt
rmdir ~/memfs_mount/docs
```

其中最后一条命令在目录非空时应失败。删除 `/docs/week1/a.txt`、`/docs/week2/a.txt`，并依次删除空目录 `/docs/week1`、`/docs/week2` 后，再执行 `rmdir ~/memfs_mount/docs` 应成功。

挑战二：简化透明加密透传文件系统

透明加密文件系统的目标是让用户在挂载点里像平常一样读写明文文件，但真实保存到磁盘上的内容是加密后的密文。这样做的典型应用包括隐私保险箱、加密同步目录、云盘客户端的本地加密层等：应用程序访问挂载点时不需要知道加密细节，文件系统在读写路径上自动完成加密和解密。

本挑战通过一个简化版本帮助你理解这种设计：挂载点提供“明文视图”，后端真实目录保存“密文数据”。这能进一步说明 FUSE 的价值：它不仅可以把内存伪装成文件系统，也可以把一个真实目录包装成带有额外行为的文件系统。

实现一个简化版 passthrough 文件系统。用户访问 FUSE 挂载点时看到的是明文；程序在后台把内容保存到一个真实目录中，但保存前对文件内容做固定规则的加密，读取时再解密。

为了控制难度，本挑战只要求支持挂载点根目录下的普通文件，不要求支持子目录、文件名加密、AES、权限继承、符号链接或目录递归。

要求：

1. 程序使用一个固定后端目录保存真实文件，例如 `~/memfs_backend`。
2. 挂载点中的普通文件操作应透传到后端目录中的同名文件。
3. 写入时对传入字节做 XOR 加密后再写入后端文件；读取时从后端文件读取密文字节，XOR 解密后再返回给用户。
4. XOR 密钥可以固定为 1 字节或短字符串，但加密后文件大小必须和明文大小相同，便于处理 offset。
5. 至少支持 `getattr`、`readdir`、`create`、`open`、`read`、`write`、`truncate`、`unlink`。
6. 在报告中说明为什么直接查看后端目录会看到密文，而通过挂载点读取会看到明文。

验收建议：

```
mkdir -p ~/memfs_backend
echo "secret" > ~/memfs_mount/a.txt
cat ~/memfs_mount/a.txt
cat ~/memfs_backend/a.txt
printf "!" >> ~/memfs_mount/a.txt
cat ~/memfs_mount/a.txt
rm ~/memfs_mount/a.txt
ls ~/memfs_backend
```

通过挂载点读取 `a.txt` 应显示明文；直接查看后端目录中的 `a.txt` 应不是原始明文。删除挂载点中的文件后，后端同名文件也应被删除。

8. 实验提交

完成实验报告的填写，以“学号-姓名-操作系统实验六”命名。

将实验报告提交到作业网站：

<https://acm.scu.edu.cn/teach/>